replacement.py — technical

`src/moe_pruning/pruning/replacement.py · 176 lines · moe_pruning.pruning.replacement`

Purpose

Generic replacement for bias / drop / reroute.

Step by step

Classes and functions in source order. Signatures, line ranges, raised exceptions and settings are extracted from the code; numbered steps describe what the code does.

`class ReplacementReport`

lines 30-47 · dataclass

Replaced experts and removed / added parameter counts.

Field	Type	Default
`replaced`	dict[str, list[int]]	field(default_factory=dict)
`removed_parameters`	int	0
`added_parameters`	int	0
`parameters_before`	int	0
`parameters_after`	int	0

`ReplacementReport.to_dict`

```
def to_dict(self) -> dict
```

lines 37-47

Convert to a plain dictionary for JSON reports.

`make_replacement_module`

```
def make_replacement_module(method: str, output_dim: int, init: torch.Tensor | None=None, dtype:
torch.dtype | None=None, device: torch.device | str | None=None) -> nn.Module
```

lines 53-67 · raises ValueError

Module installed in a removed expert's slot for the given method.

1. `bias` -> `BiasExpert(init)`; `drop` -> `ZeroExpert`; `reroute` -> `ZeroExpert(must_be_unused=True)`; else ValueError.

`replace_experts`

```
def replace_experts(model: nn.Module, adapter: MoEAdapter, layers: dict[str, MoELayerInfo], selection:
dict[str, list[int]], method: str, biases: dict[str, dict[int, torch.Tensor | None]] | None=None) ->
ReplacementReport
```

lines 70-144 · raises RuntimeError, ValueError

Remove the selected experts in place and install their replacement.

1. Validate the method (bias needs initial vectors). Parameter dtype: model dtype, promoted to float32 for 16-bit models.
2. Hash every affected router's state dict (SHA-256 of names + values).
3. Per layer: device = that layer's MoE module device; refuse experts that are already BiasExpert/ZeroExpert; build modules; `adapter.replace_experts_in_layer`; for reroute `adapter.set_routing_mask`.
4. `gc.collect()` and empty the CUDA/MPS cache so freed expert memory is returned.
5. Verify router hashes are unchanged and that `adapter.removed_weight_violations` finds nothing for all removed experts of each layer (including earlier rounds).
6. Fill and log the `ReplacementReport`.

`replace_experts_with_biases`

```
def replace_experts_with_biases(model: nn.Module, adapter: MoEAdapter, layers: dict[str, MoELayerInfo], biases: dict[str, dict[int, torch.Tensor | None]]) -> ReplacementReport
```

lines 147-155

Shorthand for `replace_experts(..., method="bias")`.

`bias_parameters`

```
def bias_parameters(model: nn.Module) -> dict[str, nn.Parameter]
```

lines 158-164

Named parameters belonging to BiasExpert modules.

`freeze_all_but_biases`

```
def freeze_all_but_biases(model: nn.Module) -> dict[str, nn.Parameter]
```

lines 167-176 · raises RuntimeError

Freeze every parameter except BiasExpert.bias and return the trainable ones.

1. Collect `BiasExpert.bias` parameters by module name; set `requires_grad` only for them; verify the trainable set equals exactly that collection.

Module constants

Name	Value
`REPLACEMENT_METHODS`	('bias', 'drop', 'reroute')

Errors raised

Function	Exception	Message (f-string fields in braces)
`make_replacement_module`	ValueError	Unknown replacement method {method}; expected one of {REPLACEMENT_METHODS}
`replace_experts`	ValueError	Unknown replacement method {method}
`replace_experts`	ValueError	The bias method needs initial bias vectors
`replace_experts`	RuntimeError	Router of {layer_id} changed during replacement
`replace_experts`	RuntimeError	Original parameters still present after replacement: {leftover}
`replace_experts`	ValueError	{layer_id} expert {expert_id} is already replaced
`freeze_all_but_biases`	RuntimeError	Unexpected trainable parameters: {actually_trainable}